

PERSONAL FINANCIAL TRACKER

A Python Console-Based Application for Recording Income, Expenses and Balance

Project Type	Python Programming Project
Application	Personal Financial Tracker
Primary Interface	Console / Command Line
Core Data Structures	Dictionary and List
Source File	manager.py

Prepared according to the VITyarthi Build Your Own Project report requirements.

1. Introduction

The Personal Financial Tracker is a Python console application for recording income, entering expenses, viewing expense records, and calculating remaining balance. The implementation uses functions, dictionaries, lists, loops, conditional statements and console I/O.

The source defines the money dictionary, expenses list and operational functions.

■filecite■turn0file0■L1-L20■

The menu provides Income, Expense, View Expense, Balance and Exit choices.

■filecite■turn0file0■L72-L100■

2. Problem Statement

Manual tracking of personal spending can make it difficult to maintain a simple running total of expenses and remaining money. This project provides a small command-line solution for entering income and expenses and calculating the current balance.

3. Objectives

- Record salary and extra income.
- Record expenses by category and amount.
- Display recorded expenses and total.
- Calculate remaining balance.
- Apply modular Python programming.

4. Functional Requirements

The VITyarthi guidelines require at least three major functional modules with clear input/output and logical workflow. ■filecite■turn0file1■L24-L37■

Module	Input	Processing	Output
Income	Salary; extra income	Stores values and adds them	Confirmation + total
Expense	Category; amount	Creates and appends record	Confirmation
View Expense	None	Iterates and totals records	Expense list + total
Balance	None	Income minus expenses	Balance summary
Exit	Choice 5	Calls sys.exit()	Exit message

Income() stores salary and extra income. ■filecite■turn0file0■L10-L18■

Expense() creates and appends an expense record. ■filecite■turn0file0■L20-L32■

View_Expense() lists records and totals them; Balance() calculates the remaining balance. ■filecite■turn0file0■L34-L64■

5. Non-Functional Requirements

- Usability: clear prompts and menu labels.
- Maintainability: operations are separated into functions.
- Resource efficiency: simple in-memory structures.
- Error handling: invalid menu choice has an explicit branch.

6. System Architecture

Console input is received by the menu controller, routed to a function, used to update in-memory data, and returned as console output.

System Architecture

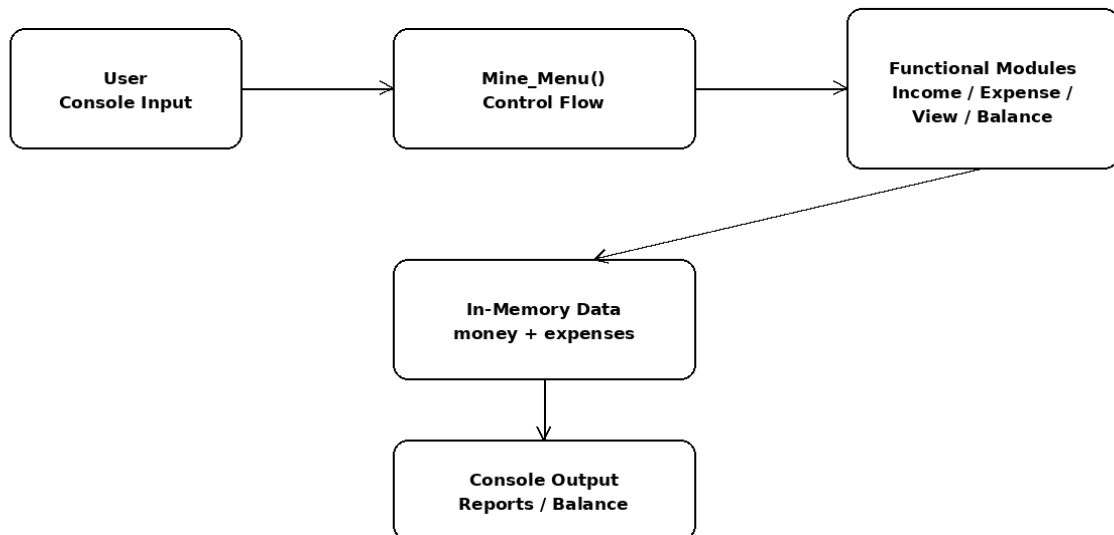


Figure 1. System architecture.

7. Design Diagrams

7.1 Use Case Diagram

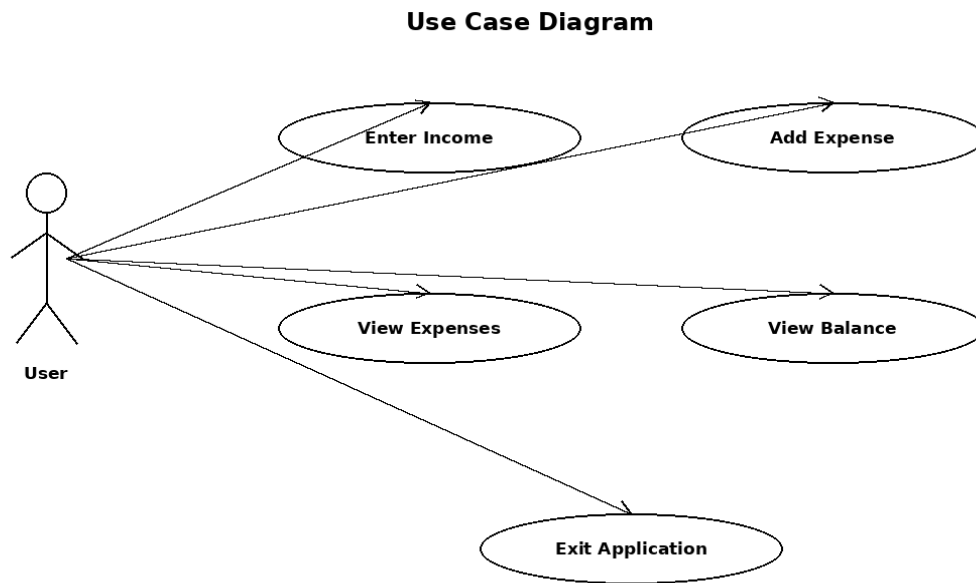


Figure 2. User interactions.

7.2 Workflow Diagram

Personal Financial Tracker - Workflow

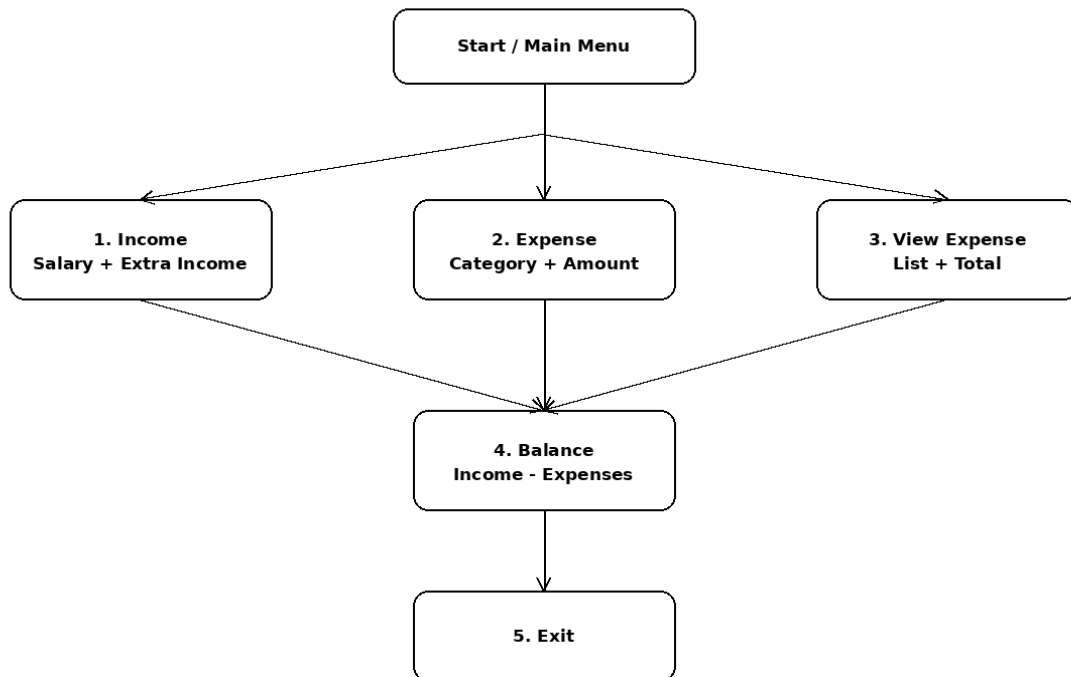


Figure 3. Main workflow.

7.3 Component / Function Diagram

Component / Function Diagram

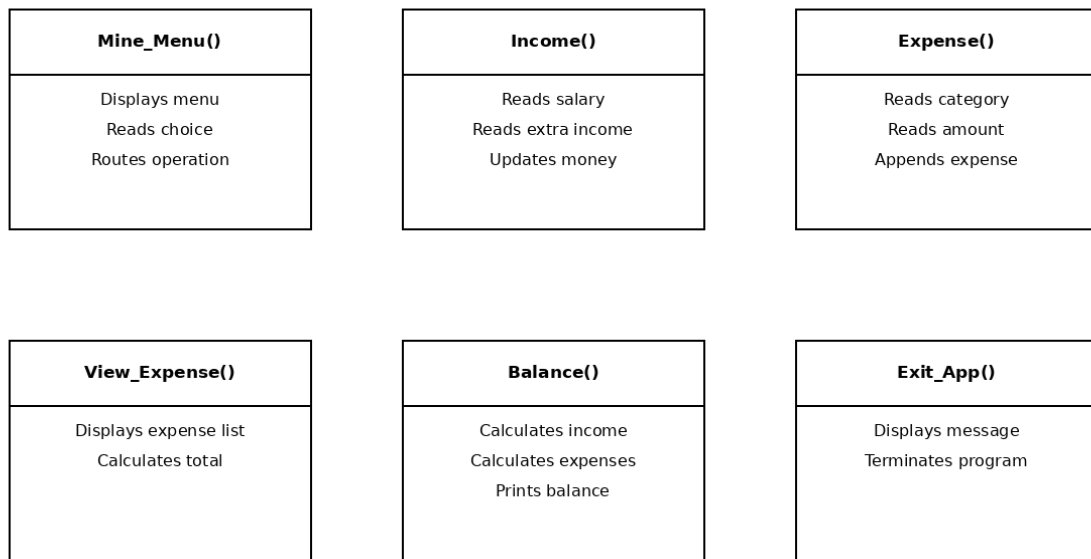


Figure 4. Major functions in manager.py.

8. Data / Storage Design

No external database is implemented. The project uses an in-memory dictionary named `money` and a list named `expenses`. The income fields are `salary` and `extra_income`, while expense records contain `category` and `amount`. ■filecite■turn0file0■L1-L8■ ■filecite■turn0file0■L21-L30■

```
money = { salary, extra_income }
expenses = [ { category, amount }, ... ]
```

9. Design Decisions & Rationale

- Functions separate major operations.
- Dictionary represents named income fields.
- List of dictionaries stores multiple expense records.
- Menu-driven console interface keeps interaction direct.
- Balance is calculated from current data.

10. Implementation Details

The source imports sys and defines the data structures before the operational functions.

■filecite■turn0file0■L1-L8■

Income Processing

Income() reads salary and extra income, stores them, calculates their sum and prints confirmation.

■filecite■turn0file0■L10-L18■

Expense Processing

Expense() accepts category and amount, creates a dictionary and appends it to expenses.

■filecite■turn0file0■L20-L32■

Expense Reporting

View_Expense() handles an empty list, prints each record and calculates total expense.

■filecite■turn0file0■L34-L52■

Balance Calculation

Balance() calculates total income, total expense and remaining balance. ■filecite■turn0file0■L54-L64■

Menu and Exit

Mine_Menu() routes choices to functions and choice 5 calls Exit_App(), which uses sys.exit().

■filecite■turn0file0■L66-L100■

11. Screenshots / Results

No runtime screenshots were supplied with the source files used for this report. The implemented result flow is Income → Expense → View Expense → Balance → Exit. The source contains confirmation, expense-list, total and balance outputs. [■filecite■turn0file0■L16-L18■](#) [■filecite■turn0file0■L30-L32■](#) [■filecite■turn0file0■L40-L52■](#) [■filecite■turn0file0■L61-L64■](#)

12. Testing Approach

Test	Action	Expected Result
TC-01	Income with numeric values	Values stored; confirmation
TC-02	Expense with category + amount	Record appended
TC-03	View with no records	NO EXPENSE!!
TC-04	View with records	Records + total
TC-05	Balance	Income, expense, balance
TC-06	Invalid menu choice	Invalid Choice
TC-07	Exit	Message + termination

The empty-expense and invalid-choice branches are explicitly implemented.

[■filecite■turn0file0■L34-L40■](#) [■filecite■turn0file0■L94-L100■](#)

13. Challenges Faced

- Menu routing.
- Managing multiple expense records in memory.
- Calculating totals correctly.
- Keeping operations modular.

14. Learnings & Key Takeaways

- Python functions and modular structure.
- Dictionaries and lists for structured data.
- Iteration and conditionals.
- Formatted console output and numeric conversion.

15. Future Enhancements

- Persistent storage.
- Stronger numeric validation.
- Edit/delete expense records.
- Dates and monthly summaries.
- Category reports and charts.
- GUI/web interface.
- Automated tests.

16. References

1. VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines (provided PDF). [filecite turn0file1 L109-L134](#)
2. Submitted Python source file: manager.py. [filecite turn0file0 L1-L102](#)

17. Project-to-Guideline Alignment

Requirement	Included
Cover Page	Yes
Introduction / Problem Statement	Yes
Functional / Non-functional Requirements	Yes
Architecture / Design Diagrams	Yes
Storage Design	Yes; in-memory
Implementation Details	Yes
Screenshots / Results	Flow documented; no runtime screenshots supplied
Testing	Yes
Challenges / Learnings / Future Enhancements	Yes
References	Yes

The VITyarthi document requires the detailed project report as a PDF and lists the report components used above. [filecite turn0file1 L109-L134](#)

End of Project Report